COFA 운영 규칙 · 분석 먼저, 3세션 협업

기본 흐름: A 분석·계획 → 사용자 검토·운영진 답변 → 명시적 구현 시작 → 병렬 구현·통합 → B 검증 → A 수정 → C 기록·복습. 실제 복붙 순서는 QUICKSTART.md를 따른다.

1. 계획 단계의 경계

처음에는 A만 실행한다. 제품 코드·의존성·DB 변경, 구현 에이전트 실행, 워크트리 생성은 “구현 시작” 이후다. 계획 문서 작성과 독립적인 읽기·분석은 가능하다. 기존 실행 검사는 격리되어 데이터 변경 위험이 없는 범위만 확인한다.

분석은 공식 요구·데이터 의미·핵심 호출 경로·제약·실패 동작을 실제 파일로 확인한다. 의존성/생성물 전수 읽기나 문서 분량을 깊이의 기준으로 삼지 않는다. 기존 계획 정본이 있으면 갱신하고 PLAN.md를 중복 생성하지 않는다.

PLAN의 첫 1쪽은 사람이 읽는 요약이다. 상세에는 요구 ID·수락 조건·현 구조·데이터 흐름·가정·미확인·로직 대안·공유 계약·담당 범위·의존성·검사·실행/제출 방법을 둔다. 중요한 7문항의 답변 근거를 어떤 코드와 검사로 확보할지도 계획한다.

고객 질문은 에이전트가 작성하고 사용자가 전달한다. 기존 자료에 답이 없는 업무 질문을 Q번호로 한 번에 1~3개 제시하며, 이유와 답에 따른 구현 차이를 붙인다. 일반 기술 선택은 에이전트가 추천한다. 실제 답변과 자체 가정을 혼동하지 않는다.

사용자가 읽는 동안 분석을 보완할 수 있으나 구현은 시작하지 않는다. 시간 경과나 고객 답변 전달만으로 구현 승인을 추정하지 않는다.

2. 세션별 책임과 파일 소유

역할	책임	편집할 공통 기록	금지/주의
A 구현·통합	계획, 코드, 작업 배정, 수정, 최종 실행	PLAN, .work/HANDOFF.md	B/C 기록 덮어쓰기 금지
B 검증	고정된 변경 묶음 재현·검사	.work/REVIEW.md	제품 코드 수정 금지
C 기록	실제 근거 기반 설명·리허설	PROJECT_DEFENSE.md	제품 코드·검사 중복 실행 금지

세션은 같은 과제 루트에서 열되 편집 파일을 구분한다. 기존 정본이 있으면 그 경로를 PLAN과 HANDOFF에 적어 모든 세션이 동일한 문서를 사용한다. 자료 저장소 .cofa-playbook 안에는 실제 과제 코드나 기록을 쓰지 않는다.

별도 세션은 대화를 자동으로 공유하지 않는다. 파일 변경도 세션을 자동 실행시키지 않는다. 사용자가 QUICKSTART의 짧은 전달 명령으로 각 세션을 재개한다. 자동 메시징 기능을 실제 지원하는 환경에서는 A가 이를 사용할 수 있지만 지원 여부를 먼저 확인한다.

3세션이 허용되지 않거나 리소스가 부족하면 A가 구현→검증→기록 순서로 수행한다. 기록 C는 중요한 통합 시점에만 실행해도 된다. 세션 수를 늘리는 것 자체가 목표가 아니다.

3. 구현 시작과 워크트리

사용자의 “구현 시작” 후 A가 공통 계약과 필요한 골격을 먼저 준비한다. 독립 작업 최대 2개를 워크트리로 나누고 지원되는 구현 서브에이전트에 배정한다. UI/API 등 이름만으로 나누지 말고 입력·응답·오류·상태 계약과 편집 범위로 나눈다.

워크트리는 기준 커밋을 공유하지만 미커밋 변경을 자동으로 가져오지 않는다. 기존 변경을 보존하고 필요한 코드·계약이 기준에 포함됐는지 먼저 확인한다. 공식 규칙에 맞춰 필요한 변경만 명시적으로 포함하며 무조건 전체 파일을 커밋하지 않는다.

작업자에게 요구 ID, 목표, 정확한 경로·브랜치·기준, 읽을 파일, 편집 허용 범위, 공통 계약, 수락 조건, 검사, 반환할 근거를 준다. 작업자는 추가 에이전트를 무제한 생성하지 않는다.

공유 스키마·의존성·락파일·마이그레이션은 한 담당자가 소유한다. 테스트 DB·포트·출력 경로를 분리하고 운영/공유 DB를 검사용으로 수정하지 않는다. API 계약 변경은 A가 영향받는 담당자에게 전달하고 수신·반영을 확인한다.

작업자는 변경 파일·함수·선택 이유·검사 명령/결과·한계·통합할 변경을 반환한다. A는 실제 제출 루트에 순차 통합하고 연결 검사를 수행한다. 첫 입력→최종 결과 흐름을 조기에 확인한 뒤 필수 요구를 모두 완성한다. 워크트리가 불가능하면 파일별 단독 담당 또는 순차 작업으로 대체한다.

4. 검증과 수정 루프

A는 첫 핵심 흐름과 최종 통합 시점에 검토 가능한 고정 묶음을 만든다. HANDOFF에 검토 ID·기준 커밋 또는 파일 지문·격리 경로·변경 파일·요구/수락 조건·재현 입력·검사 명령을 기록한다. B가 보는 사본은 검토 중 변경하지 않는다.

B는 실행 가능한 격리 환경에서 검사하고 REVIEW에 실제 결과를 남긴다. 기준이 없거나 변하는 파일만 있으면 재현 가능한 기준을 요청한다. 요구 누락·계산 오류·검증 실패 데이터 사용·조건/DB 변경·중간 실패·데이터 유실·실행 실패를 우선한다.

지적은 검토 ID·버전·요구 ID·심각도·파일:줄·재현 절차·기대/실제·최소 수정안을 포함한다. 코드 해석만으로 추정한 우려와 실제 재현을 구분한다. 취향 리팩터링과 필수 결함은 분리한다.

A가 지적을 대조하고 원 구현자가 수정한다. 처리 근거와 새 검토 기준을 HANDOFF에 남긴다. B가 새 기준에서 같은 사례와 영향 범위를 재확인한다. 검사 통과를 위해 수락 기준을 약화하거나 검사를 삭제하지 않는다.

검토한 버전이 현재 코드와 다르면 과거 통과를 최신 통과로 쓰지 않는다. 최종 검사는 제출 루트에서도 실행한다. UI 도구가 없으면 사용자가 수행할 화면 확인 절차를 제시하고 미확인 화면을 정상이라고 기록하지 않는다.

5. 기록과 핵심 질문 복습

C는 docs/핵심질문7개.md를 기준으로 실제 구현 설명을 정리한다. 설명은 현재 동작 → 선택 이유 → 파일/함수 → 실제 검사/수정 → 한계 순서로 짧게 쓴다. 배포 방식도 함께 정리한다.

핵심: 알고리즘과 최적성, 외부 패키지/내부 모듈, 조건 변경, 검증 중 고친 사례, 한 건의 데이터 흐름, 추가 가정과 검증 실패 데이터, 계산 중 오류/DB 변경.

배포 설정 파일과 실제 배포 성공, async 문법과 내구성 있는 작업 큐, 실행 가능해와 최적해를 구분한다. 구현하지 않은 장치를 향후 개선안으로 쓸 수 있지만 현재 기능으로 주장하지 않는다.

C는 A/B 보고와 실제 코드·로그를 대조한다. 검사 재실행 없이 근거를 읽고, 참조한 검토 ID/버전과 작성 시점을 기록한다. 코드가 바뀌면 해당 설명을 재확인 필요로 표시한다. 실제 발생하지 않은 버그 수정 사례를 꾸며내지 않는다.

사용자는 허용된 사전 연습 시간에 한 문제씩 답한다. C는 정답을 먼저 보여주지 않고 답변을 코드와 대조한다. 실제 Q&A의 AI 제한은 현재 공식 규정을 따른다.

6. 시간·진행 보고·재개

예시: 180분이면 처음 20~30분은 분석·계획·검토, 마지막 30~35분은 최종 검증·제출 확인·복습에 남긴다. 공식 제한이 아닌 운영 예시이며 실제 난이도·남은 시간에 맞춘다. 계획 검토 전에는 시간이 지났다는 이유로 구현을 시작하지 않는다.

A는 작업 중 대략 5분마다 완료한 요구·검사 근거·막힘·다음 행동을 보고한다. 완료·실패·계약 변경은 즉시 알린다. 별도 세션을 예약 실행하거나 정확히 5분마다 깨우는 기능을 보장하지 않는다.

세션 재개 시 역할, 계획 경로, HANDOFF·REVIEW·설명 정본과 실제 변경을 대조한다. 계획 단계인지 구현 시작 이후인지 확인하고, 상태를 모르는 워커·서버는 중복 실행 전에 조사한다.

미답변 업무 규칙은 영향받는 작업만 막는다. 계획 단계에서는 독립 분석을, 구현 시작 이후에는 독립 구현을 계속한다. 단, 조건을 추측해 고객의 확정 답변으로 기록하지 않는다.

최종 필수 요구·큰 결함·실행/저장 실패를 우선한다. 미완료를 명시하고 사용자가 직접 최종 화면·저장·제출을 확인할 시간을 확보한다. 별도 세션·워크트리 운영 비용이 크면 A에서 순차 진행한다.

참고: 구현 문답 관점은 https://github.com/leey00nsu/cofa-hackathon-kit, 독립 작업 분할 관점은 https://github.com/obra/superpowers 를 참고했다. 이 운영안은 공식 시험 지침이 아닌 개인 준비용 제안이다.